

- Description

commons-math-4.0-beta1-src\commons-math-
legacy\src\main\java\org\apache\commons\math4\legacy\analysis\solvers\
RiddersSolver.java

- Specification

module RiddersSolver

type

min, max, start: real // search interval [min, max] and optional start
maxEval: integer // maximum allowed function evaluations
relAcc, absAcc, fAcc: real // relative, absolute, function-value tolerances
r: real // returned root
ex: set of {None, NullArgument, BadSequence, NoBracketing, TooManyEvaluations}

process solve(f, min, max, start, maxEval, relAcc, absAcc, fAcc: real) r: real, ex: set of {None, NullArgument, BadSequence, NoBracketing, TooManyEvaluations}

functional scenario form:

```
f = null && ex = NullArgument
||
min > max && ex = BadSequence
||
bracketed = 0 && ex = NoBracketing
||
yMin = 0 && ex = None && r = min
||
yMax = 0 && ex = None && r = max
||
yStart = 0 && ex = None && r = start
||
yStart * yMin < 0 && ex = None
    && r ∈ (min, start] && |f(r)| <= fAcc
||
yStart * yMax < 0 && ex = None
    && r ∈ [start, max) && |f(r)| <= fAcc
||
b - a <= tol
    && ex = None
    && r = xR && |f(r)| <= fAcc
||
a'*b'<0 && ex = None
    && r ∈ [a', b'] // continues iterations until a termination condition holds
||
evaluations > maxEval && ex = TooManyEvaluations
```

end process
end module

- Java Code

```
/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.analysis.solvers;
import org.apache.commons.math4.legacy.exception.NoBracketingException;
import org.apache.commons.math4.legacy.exception.TooManyEvaluationsExce
ption;
import org.apache.commons.math4.core.jdkmath.JdkMath;
/**
 * Implements the <a href="http://mathworld.wolfram.com/RiddersMethod.h
tml">
 * Ridders' Method</a> for root finding of real univariate functions. F
or
 * reference, see C. Ridders, <i>A new algorithm for computing a single
root of
 * a real continuous function </i>, IEEE Transactions on Circuits and S
ystems,
 * 26 (1979), 979 - 980.
 * <p>
 * The function should be continuous but not necessarily smooth.
 * </p>
 *
 * @since 1.2
 */
```

```

public class RiddersSolver extends AbstractUnivariateSolver {
    /** Default absolute accuracy. */
    private static final double DEFAULT_ABSOLUTE_ACCURACY = 1e-6;
    /**
     * Construct a solver with default accuracy (1e-6).
     */
    public RiddersSolver() {
        this(DEFAULT_ABSOLUTE_ACCURACY);
    }
    /**
     * Construct a solver.
     *
     * @param absoluteAccuracy Absolute accuracy.
     */
    public RiddersSolver(double absoluteAccuracy) {
        super(absoluteAccuracy);
    }
    /**
     * Construct a solver.
     *
     * @param relativeAccuracy Relative accuracy.
     * @param absoluteAccuracy Absolute accuracy.
     */
    public RiddersSolver(double relativeAccuracy, double absoluteAccuracy)
    {
        super(relativeAccuracy, absoluteAccuracy);
    }
    /**
     * {@inheritDoc}
     */
    @Override
    protected double doSolve() throws TooManyEvaluationsException, NoBracketingException {
        double min = getMin();
        double max = getMax();
        // [x1, x2] is the bracketing interval in each iteration
        // x3 is the midpoint of [x1, x2]
        // x is the new root approximation and an endpoint of the new interval
        double x1 = min;
        double y1 = computeObjectiveValue(x1);
        double x2 = max;
        double y2 = computeObjectiveValue(x2);
        // check for zeros before verifying bracketing

```

```

if (y1 == 0) {
    return min;
}
if (y2 == 0) {
    return max;
}
verifyBracketing(min, max);
final double absoluteAccuracy = getAbsoluteAccuracy();
final double functionValueAccuracy = getFunctionValueAccuracy();
final double relativeAccuracy = getRelativeAccuracy();
double oldx = Double.POSITIVE_INFINITY;
while (true) {
    // calculate the new root approximation
    final double x3 = 0.5 * (x1 + x2);
    final double y3 = computeObjectiveValue(x3);
    if (JdkMath.abs(y3) <= functionValueAccuracy) {
        return x3;
    }
    final double delta = 1 - (y1 * y2) / (y3 * y3); // delta > 1 due to
bracketing
    final double correction = (JdkMath.signum(y2) * JdkMath.signum(y3))
* (x3 - x1) / JdkMath.sqrt(delta);
    final double x = x3 - correction; // correction != 0
    final double y = computeObjectiveValue(x);
    // check for convergence
    final double tolerance = JdkMath.max(relativeAccuracy * JdkMath.abs(
x), absoluteAccuracy);
    if (JdkMath.abs(x - oldx) <= tolerance) {
        return x;
    }
    if (JdkMath.abs(y) <= functionValueAccuracy) {
        return x;
    }
    // prepare the new interval for next iteration
    // Ridders' method guarantees x1 < x < x2
    if (correction > 0.0) { // x1 < x < x3
        if (JdkMath.signum(y1) + JdkMath.signum(y) == 0.0) {
            x2 = x;
            y2 = y;
        } else {
            x1 = x;
            x2 = x3;
            y1 = y;
            y2 = y3;
        }
    }
}

```

```

    }
} else { // x3 < x < x2
    if (JdkMath.signum(y2) + JdkMath.signum(y) == 0.0) {
        x1 = x;
        y1 = y;
    } else {
        x1 = x3;
        x2 = x;
        y1 = y3;
        y2 = y;
    }
}
oldx = x;
}
}
}
//VT
class RSVT {
    private static boolean approx(double a, double b, double tol) {
        if (Double.isNaN(a) || Double.isNaN(b))
            return false;
        if (Double.isInfinite(a) || Double.isInfinite(b))
            return a == b;
        return Math.abs(a - b) <= tol * Math.max(1.0, Math.max(Math.abs(a), Math.abs(b)));
    }
    private static void check(String name, boolean cond) {
        System.out.printf("[%s] %s\n", cond ? "PASS" : "FAIL", name);
    }
    private static void runCase(String name, org.apache.commons.math4.legacy.analysis.UnivariateFunction f, double min,
        double max, int maxEval) {
        System.out.println("---- " + name + " (no start) ----");
        try {
            RiddersSolver solver = new RiddersSolver(); // default tolerances
            double r = solver.solve(maxEval, f, min, max);
            System.out.printf("root=%.17g, f(root)=%.3e\n", r, f.value(r));
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
        }
    }
    private static void runCaseWithStart(String name, org.apache.commons.math4.legacy.analysis.UnivariateFunction f,

```

```

        double min, double max, double start, int maxEval) {
System.out.println("---- " + name + " (with start) ----");
try {
    RiddersSolver solver = new RiddersSolver();
    double r = solver.solve(maxEval, f, min, max, start);
    System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r));
} catch (Throwable t) {
    System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
}
}

// Custom solver (e.g., to nudge step-
size termination with a looser absAcc)
private static void runCaseCustom(String name, org.apache.commons.math
4.legacy.analysis.UnivariateFunction f,
    double min, double max, int maxEval, double relAcc, double absAcc, d
ouble fAcc) {
    System.out.println("---- " + name + " (custom tolerances) ----");
    try {
//        RiddersSolver solver = new RiddersSolver(relAcc, absAcc, fAcc
);
//        double r = solver.solve(maxEval, f, min, max);
//        System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r)
);
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

public void Vtest() {
    System.out.println("=== RiddersSolver V-method tests ===");
    // VT1: f == null → NullArgument
    System.out.println("---- VT1 f==null ----");
    try {
        RiddersSolver solver = new RiddersSolver();
        solver.solve(100, null, -1.0, 1.0);
        check("VT1 should throw NullArgument", false);
    } catch (IllegalArgumentException ok) {
        System.out.println("threw: " + ok.getClass().getSimpleName());
        check("VT1 threw (NullArgument)", true);
    } catch (Throwable t) {
        t.printStackTrace();
        check("VT1 wrong exception", false);
    }
}

```

```

// VT2: min > max → BadSequence
System.out.println("---- VT2 min>max ----");
try {
    RiddersSolver solver = new RiddersSolver();
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
x - 1.0;
    solver.solve(100, f, 2.0, -2.0);
    check("VT2 should throw BadSequence", false);
} catch (IllegalArgumentException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    check("VT2 threw (BadSequence)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("VT2 wrong exception", false);
}
// VT3: No bracketing → should throw
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
x * x + 1.0; // >0 on all reals
    new RiddersSolver().solve(100, f, -3.0, 2.0);
    check("VT3 should throw NoBracketing", false);
} catch (IllegalArgumentException ok) {
    System.out.println("VT3 threw: " + ok.getClass().getSimpleName());
    check("VT3 threw (NoBracketing)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("VT3 wrong exception", false);
}
// VT4: endpoint root at min
runCase("VT4 endpoint min root", x -> x + 2.0, -
2.0, 3.0, 200); // root at -2
// VT5: endpoint root at max
runCase("VT5 endpoint max root", x -> x - 4.0, 2.0, 4.0, 200); // roo
t at 4
// VT6: start in interval & yStart*max < 0 → triage to [start, max)
// min=1.6, start=2.8 (f<0), max=3.5 (f>0) → expect ~3
runCaseWithStart("VT6 start*max opposite sign", x -> (x - 1.0) * (x -
3.0), 1.6, 3.5, 2.8, 400);
// VT7: regular convergence inside bracket (monotone cubic near root)
runCase("VT7 regular convergence", x -> (x - 2.0) * (x - 5.0) * (x +
1.0), 1.5, 2.6, 400); // expect ~2
// VT8
runCase("VT8 opposite to y1 → update right", x -> Math.expm1(12.0 * x
) - Math.expm1(9.6), 0.0, 1.0, 400);

```

```

// VT9: TooManyEvaluations on bracketed root (tiny budget)
System.out.println("---- VT9 TooManyEvaluations ----");
try {
    RiddersSolver solver = new RiddersSolver();
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
Math.cos(x) - x; // root ~0.739
    solver.solve(1, f, 0.0, 1.0, 0.5);
    check("VT9 should throw TooManyEvaluations", false);
} catch (IllegalStateException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    check("VT9 threw (TooManyEvaluations)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("VT9 wrong exception", false);
}
// VT10: sharp nonlinearity to exercise different Ridders probe place
ments
runCase("VT10 sharp nonlinearity", x -> Math.atan(50.0 * x), -
0.5, 0.5, 500); // root at 0 (excluded but
// bracketed)
// VT11: near-linear function to probe tolerance logic further
runCase("VT11 near linear", x -> 1e-
9 * x + (x - 0.7), 0.2, 1.2, 400); // expect ~0.7
System.out.println("=== Done RiddersSolver V-method tests ===");
}
}
//FT
//VT
class RSFT {
    private static boolean approx(double a, double b, double tol) {
        if (Double.isNaN(a) || Double.isNaN(b))
            return false;
        if (Double.isInfinite(a) || Double.isInfinite(b))
            return a == b;
        return Math.abs(a - b) <= tol * Math.max(1.0, Math.max(Math.abs(a), M
ath.abs(b)));
    }
    private static void check(String name, boolean cond) {
        System.out.printf("[%s] %s\n", cond ? "PASS" : "FAIL", name);
    }
    private static void runCase(String name, org.apache.commons.math4.lega
cy.analysis.UnivariateFunction f, double min,
        double max, int maxEval) {
        System.out.println("---- " + name + " (no start) ----");

```

```

try {
    RiddersSolver solver = new RiddersSolver(); // default tolerances
    double r = solver.solve(maxEval, f, min, max);
    System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r));
} catch (Throwable t) {
    System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
}
}

private static void runCaseWithStart(String name, org.apache.commons.m
ath4.legacy.analysis.UnivariateFunction f,
    double min, double max, double start, int maxEval) {
    System.out.println("---- " + name + " (with start) ----");
    try {
        RiddersSolver solver = new RiddersSolver();
        double r = solver.solve(maxEval, f, min, max, start);
        System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r));
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

// Custom solver (e.g., to nudge step-
size termination with a looser absAcc)
private static void runCaseCustom(String name, org.apache.commons.math
4.legacy.analysis.UnivariateFunction f,
    double min, double max, int maxEval, double relAcc, double absAcc, d
ouble fAcc) {
    System.out.println("---- " + name + " (custom tolerances) ----");
    try {
//      RiddersSolver solver = new RiddersSolver(relAcc, absAcc, fAcc);
//      double r = solver.solve(maxEval, f, min, max);
//      System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r));
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

public void Ftest() {
    System.out.println("=== RiddersSolver Fuzzing tests ===");
// FT01:
    runCaseWithStart("FT01", x -> x - 0.6679011001, -
2.337110022, 3.119900177, 0.6679011001, 400);
// FT02:

```

```

    runCaseWithStart("FT02", x -> Math.sin(4.66920160910299 * x) - 0.0011
772019, -0.1190013377, 0.0917720011,
    0.013370022, 600);
// FT03:
    runCase("FT03", x -> Math.exp(290.331901177 * x) - 3.832023139620673E
126, 0.6679011001, 1.33722001003, 350);
// FT04:
    runCase("FT04", x -> Math.tanh(57.9011772001 * x) - 0.000003311177, -
0.00337110221, 0.00223190011, 280);
// FT05:
    runCaseWithStart("FT05", x -> x - 1.117720119 / (1.0 + 0.003901223117
* x + 2.2e-9 * x * x), -3.337200119,
    4.66920160910299, -0.773102211903, 700);
// FT06:
    runCase("FT06", x -> (x - 0.883002170113) * (x + 0.77119001127) * (x
- 1.33722001003), -2.337110022,
    3.119900177, 650);
// FT07:
    runCase("FT07", x -> Math.cos(7.0 * x + 0.22310011) - 0.00331177, -
1.70033119001, 1.70022310011, 420);
// FT08:
    runCaseWithStart("FT08", x -> Math.tan(0.77119 * x) - 0.01 * x, -
3.337110022, 3.119900177, 0.6679011001, 550);
// FT09:
    runCase("FT09", x -> Math.atan(9.91177201 * x) + 0.001 * x - 0.331700
22, -99.1177201, 109.331900177, 700);
// FT10:
    runCaseWithStart("FT10", x -> Math.expm1(60.002 * x) - 1.0e-7, -
0.331190011, 0.291772001, 0.0231900117, 360);
// FT11:
    runCase("FT11", x -> Math.exp(-
0.3317002 * Math.abs(x)) * Math.sin(33.1177201 * x) - 0.00091, -
5.119001,
    4.6692016091, 480);
    System.out.println("=== Done RiddersSolver Fuzzing tests ===");
}
}
// Z3
class RSZ3 {
    private static void check(String name, boolean cond) {
        System.out.printf("[%s] %s\n", cond ? "PASS" : "FAIL", name);
    }
    private static void runCase(String name, org.apache.commons.math4.lega
cy.analysis.UnivariateFunction f, double min,

```

```

        double max, int maxEval) {
System.out.println("---- " + name + " ----");
try {
    RiddersSolver solver = new RiddersSolver();
    double r = solver.solve(maxEval, f, min, max);
    System.out.printf("root=%.17g, f(root)=%.3e\n", r, f.value(r));
} catch (Throwable t) {
    System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
}
}

private static void runCaseWithStart(String name, org.apache.commons.m
ath4.legacy.analysis.UnivariateFunction f,
    double min, double max, double start, int maxEval) {
System.out.println("---- " + name + " (with start) ----");
try {
    RiddersSolver solver = new RiddersSolver();
    double r = solver.solve(maxEval, f, min, max, start);
    System.out.printf("root=%.17g, f(root)=%.3e\n", r, f.value(r));
} catch (Throwable t) {
    System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
}
}

public void Z3test() {
System.out.println("=== RiddersSolver 11 Z3-derived scenarios ===");
// 1) f=null -> ex=NullArgument
System.out.println("---- S1: f=null -> NullArgument ----");
try {
    new RiddersSolver().solve(100, null, -1.0, 1.0);
    check("S1 should throw NullArgument", false);
} catch (IllegalArgumentException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    check("S1 threw (NullArgument)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("S1 wrong exception", false);
}
// 2) min>max -> ex=BadSequence (min=0, max=-1/2)
System.out.println("---- S2: min>max -> BadSequence ----");
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
x - 1.0;
    new RiddersSolver().solve(100, f, 0.0, -0.5);

```

```

    check("S2 should throw BadSequence", false);
} catch (IllegalArgumentException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    check("S2 threw (BadSequence)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("S2 wrong exception", false);
}
// 3) not bracketed -> ex=NoBracketing (min=-999, max=0)
// Use  $f(x)=x^2+1$  ( $>0$  on all reals) to violate bracketing.
System.out.println("---- S3: not bracketed -> NoBracketing ----");
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
x * x + 1.0;
    new RiddersSolver().solve(100, f, -999.0, 0.0);
    check("S3 should throw NoBracketing", false);
} catch (IllegalArgumentException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    check("S3 threw (NoBracketing)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("S3 wrong exception", false);
}
// 4) yMin=0 -> ex=None, r=min (min=0, max=1, r=0)
runCase("S4: endpoint root at min", x -> x, 0.0, 1.0, 200); //  $f(0)=0$ 
→ r=min
// 5) yMax=0 -> ex=None, r=max (min=0, max=1, r=1)
runCase("S5: endpoint root at max", x -> x - 1.0, 0.0, 1.0, 200); //
 $f(1)=0$  → r=max
// 6) isIn(start) & yStart=0 -> ex=None, r=start (min=-
1, max=0, start=0, r=0)
runCaseWithStart("S6: start is root", x -> x, -
1.0, 0.0, 0.0, 200); //  $f(start)=0$  → r=start
// 7) start*min opposite sign -> ex=None,  $r \in (min, start]$ ,  $|f(r)| \leq fAcc$ 
// Choose  $r=1/2$ ; take  $f(x)=x-$ 
1/2, min=0, start=1, pick max=2 (any  $>1$  is fine).
runCaseWithStart("S7: start*min opposite sign (r in (min,start])", x
-> x - 0.5, 0.0, 2.0, 1.0, 200); // expect
// r=0.5
// 8) start*max opposite sign -> ex=None,  $r \in [start, max)$ ,  $|f(r)| \leq fAcc$ 
//  $r=1/2$ ; take  $f(x)=x-1/2$ , start=0, max=1, choose min=-
1 (any  $<0$  is fine).
runCaseWithStart("S8: start*max opposite sign (r in [start,max))", x
-> x - 0.5, -1.0, 1.0, 0.0, 200); // expect

```

```

// r=0.5
// 9)  $\exists xR: (|f(xR)| \leq fAcc \text{ or } |b-a| \leq tol) \rightarrow ex=None, r=xR$ 
// Use  $r=xR=-1$  with bracket  $[-2,0]$ ,  $f(x)=x+1$ .
runCase("S9: exists xR  $\rightarrow r=xR$ ", x -> x + 1.0, -
2.0, 0.0, 200); // root -1
// 10)  $\exists(a',b')$  narrowing  $\rightarrow ex=None, r \in [a',b']$ 
// We simply set a clean root  $r=5/4$  in  $[0,3]$ :  $f(x)=x-1.25$ 
runCase("S10: narrowed bracket contains r", x -> x - 1.25, 0.0, 3.0,
200); // root 1.25
// 11) evaluations > maxEval  $\rightarrow ex=TooManyEvaluations$ 
// Pass a negative budget (maxEval=-
1) on a bracketed root  $\rightarrow$  immediately
// triggers budget path.
System.out.println("---- S11: TooManyEvaluations (maxEval=-1) ----");
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
Math.cos(x) - x; // bracket [0,1]
    new RiddersSolver().solve(-1, f, 0.0, 1.0);
    check("S11 should throw TooManyEvaluations", false);
} catch (IllegalStateException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    check("S11 threw (TooManyEvaluations)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("S11 wrong exception", false);
}
System.out.println("=== Done 11 scenarios ===");
}
}
class RStest {
    public static void main(String[] args) {
        // VT
        RSVT t1 = new RSVT();
        t1.Vtest();
        // FT
        RSFT t2 = new RSFT();
        // t2.Ftest();
        // Z3
        RSZ3 t3 = new RSZ3();
        // t3.Z3test();
    }
}

```

- Coverage report

1. Test case number: 11
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ RiddersSolver.java	37.0 %	17	29	46
> RSFT	0.0 %	0	14	14
> RSZ3	0.0 %	0	2	2
> RSVT	7.1 %	1	13	14
▼ RiddersSolver	100.0 %	16	0	16
◆ doSolve()	100.0 %	16	0	16
RiddersSolver()		0	0	0
RiddersSolver(double)		0	0	0
> RStest		0	0	0

3. V-Method testing instruction coverage: 97.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ RiddersSolver.java	32.1 %	460	971	1,431
> RSZ3	0.8 %	3	387	390
> RSFT	0.8 %	3	371	374
> RSVT	56.4 %	265	205	470
> RStest	83.3 %	15	3	18
▼ RiddersSolver	97.2 %	174	5	179
RiddersSolver()	100.0 %	4	0	4
RiddersSolver(double)	100.0 %	4	0	4
◆ doSolve()	100.0 %	166	0	166

4. Fuzz testing branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ RiddersSolver.java	19.6 %	9	37	46
> RSFT	0.0 %	0	14	14
> RSVT	0.0 %	0	14	14
▼ RiddersSolver	50.0 %	8	8	16
◆ doSolve()	50.0 %	8	8	16
RiddersSolver()		0	0	0
RiddersSolver(double)		0	0	0
> RSZ3	50.0 %	1	1	2
> RStest		0	0	0

5. Fuzz testing instruction coverage: 68.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ RiddersSolver.java	26.3 %	376	1,055	1,431
> RSVT	0.6 %	3	467	470
> RSFT	0.8 %	3	371	374
> RSZ3	59.7 %	233	157	390
▼ RiddersSolver	68.2 %	122	57	179
◆ doSolve()	68.7 %	114	52	166
RiddersSolver()	100.0 %	4	0	4
RiddersSolver(double)	100.0 %	4	0	4
> RStest	83.3 %	15	3	18

6. Z3 branch coverage: 81.2%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ RiddersSolver.java	28.3 %	13	33	46
> RSFT	0.0 %	0	14	14
> RSVT	0.0 %	0	14	14
> RSZ3	0.0 %	0	2	2
▼ RiddersSolver	81.2 %	13	3	16
◆ doSolve()	81.2 %	13	3	16
RiddersSolver()		0	0	0
RiddersSolver(double)		0	0	0
> RStest		0	0	0

7. Z3 instruction coverage: 93.9%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼  RiddersSolver.java		33.1 %	474	957	1,431
>  RSVT		0.6 %	3	467	470
>  RSZ3		0.8 %	3	387	390
>  RSFT		76.2 %	285	89	374
>  RStest		83.3 %	15	3	18
▼  RiddersSolver		93.9 %	168	11	179
 doSolve()		96.4 %	160	6	166
 RiddersSolver()		100.0 %	4	0	4
 RiddersSolver(double)		100.0 %	4	0	4